

CITS5507 Assignment 1

Parallel Birthday Attack on toy_hash — Report

Nan Xiang(25053306)

1. Algorithm and Collision-Detection Data Structure

toy_hash produces a 48-bit digest, so by the birthday paradox an expected $\sqrt{2^{48}} \approx 2^{24} \approx 16.8$ million trials on each file are needed before a matching pair of hashes is likely to appear, rather than 2^{48} trials for a fixed-target search. This is why the nonce in both files may be varied: the search compares two growing sets of hashes against each other instead of hunting for one fixed value.

The implementation therefore proceeds in two phases. Phase 1 computes `toy_hash(file_a)` for `nonce_a = 0, 1, 2, ...` and inserts each (hash, nonce_a) pair into a shared open-addressing hash table using linear probing. Phase 2 computes `toy_hash(file_b)` for `nonce_b = 0, 1, 2, ...` and probes the same table; the first match yields a valid collision. To keep probe chains short, the table has 2^{26} (≈ 67.1 million) slots, and insertion is capped at 90% occupancy (≈ 60.4 million entries) rather than 100%, since a fully saturated open-addressing table degrades linear probing from $O(1)$ to effectively $O(n)$ and can loop indefinitely once every slot is occupied — a bug we encountered and fixed during development. Note that 60.4 million is roughly $3.6\times$ the strict birthday-bound minimum of 16.8 million; this over-provisioning is a deliberate memory/time trade-off discussed in Section 3.

2. Parallelisation and Synchronisation Strategy

Both phases are parallelised with `#pragma omp parallel` for using static scheduling, since every iteration performs an identical amount of work (one `set_nonce` call and one `toy_hash` call), so static scheduling minimises scheduling overhead without causing load imbalance. Two race conditions had to be managed. First, all threads in Phase 1 mutate the same `file_a` byte buffer via `set_nonce`; each thread therefore receives its own private copy of the buffer (allocated once per thread at the start of the parallel region and freed at the end), so threads never overwrite each other's nonce bytes. The same is done for `file_b` in Phase 2.

Second, and more importantly, multiple threads insert into the same shared hash table concurrently. A single global lock (or `#pragma omp critical`) around every insertion was tried first, but it serialises all table access and becomes the dominant bottleneck as thread count grows, since Amdahl's Law shows that even a small serial fraction caps achievable speedup. The final design instead partitions the table into 128 independent partitions, selected by the low 7 bits of the 48-bit hash, each guarded by its own `omp_lock_t`. Two threads only contend for the same lock if their nonces happen to hash into the same 1-of-128 partition, so most insertions proceed fully in parallel. Phase 2 lookups use the same partition scheme; because Phase 1 has already completed by the time Phase 2 begins, table contents are stable and lookups need no additional locking. A shared `found_flag`, updated with atomic read/write and checked at the top of each loop iteration, lets every thread stop early as soon as any thread discovers a match, avoiding wasted work once a collision has been found.

3. Memory Usage and Trade-offs

Each hash-table entry stores a 64-bit hash, a 64-bit nonce and an occupancy flag, occupying 24 bytes after alignment. With 2^{26} slots the table alone occupies $2^{26} \times 24$ bytes ≈ 1.5 GiB, which matches the 1.5–1.6 GB maximum resident set size (MaxRSS) recorded by Slurm's `sacct` across all runs. This footprint is fixed regardless of file size or thread count; the only additional per-thread cost is one private copy of the input file (at most ~ 0.9 MB for $6_{\text{exa}} \times 96$ threads ≈ 88 MB), negligible next to the table itself. Given Kaya

nodes provide roughly 1.5 TB of memory, this design comfortably trades memory for speed: sizing the table at $\sim 4\times$ the expected number of insertions keeps the load factor below 90% and probe chains short, at the cost of Phase 1 performing about $3.6\times$ more hashing work than the theoretical minimum of ~ 16.8 million trials. A smaller table sized closer to the birthday bound would reduce Phase 1 time but would need careful handling of the case where a collision has not yet occurred and more insertions are required — a more complex, dynamically resized design that was judged not worth the added risk of the infinite-loop failure mode already observed once during development.

4. Performance Results and Analysis

All six provided file pairs were solved and independently verified against the Python `check_toy_hash.py` reference. Wall-clock search time (search only, excluding file I/O) for each pair across thread counts, measured on a single Kaya node, is summarised below (TIMEOUT = did not finish within the allotted Slurm time limit):

File pair (size)	1	2	4	8	16	32	48	64	96
1_kilo (65,536 B)	1:05:25	33:01	16:42	8:36	4:19	2:06	1:37	1:05	0:45
2_mega (131,072 B)	TIMEOUT	1:07:04	34:23	18:10	8:24	4:06	2:52	2:06	1:25
3_giga (229,376 B)	—	TIMEOUT	1:05:21	33:30	17:30	7:51	4:58	4:13	2:36
4_tera (344,064 B)	—	—	TIMEOUT	43:03	21:49	11:10	7:29	5:25	3:57
5_peta (491,520 B)	—	—	—	1:03:17	32:01	15:18	11:27	7:50	5:13
6_exa (921,600 B)	—	—	—	TIMEOUT	57:47	29:13	19:36	15:04	10:07

Two trends stand out. First, search time scales almost exactly linearly with file size, as expected since `toy_hash` processes every byte sequentially: at 8 threads, `4_tera` ($5.25\times$ the size of `1_kilo`) took 2583 s versus a predicted $516 \times 5.25 \approx 2709$ s (5% deviation), and `5_peta` ($7.5\times$) took 3797 s versus a predicted 3870 s (2% deviation) — strong evidence the implementation has no hidden inefficiency beyond the expected per-byte hashing cost. Second, the partitioned-lock design scales well with thread count: for `6_exa`, increasing from 16 to 96 threads ($6\times$ the cores) reduced time from 3467 s to 607 s, a $5.7\times$ speedup (95% parallel efficiency); doubling from 32 to 64 threads gave 1753 s $\rightarrow$ 904 s ($1.94\times$ speedup, 97% efficiency). Efficiency drops only slightly at the highest thread counts, consistent with a small but non-zero serial fraction (table allocation, thread spawning, and lock contention within partitions when multiple nonces hash to the same bucket).

All six pairs complete comfortably within the assignment's 15-minute limit at 64 or 96 threads; the low-thread-count TIMEOUTs on the larger pairs are themselves informative, showing precisely why parallel execution on Kaya is necessary rather than optional for this workload — `1_kilo` alone is estimated to need 65 minutes on a single core.